

Security by Separation, Verification, and Settlement

An AMLD-Oriented Mathematical Treatise

Brian Tenneson

Working research manuscript v0.1

August 30, 2026

This first draft is intentionally broader and more elementary than the password-domain material in *What Checks the Proof?* v6.80. It begins with information, secrecy, separation, identity, threat models, and containment, then introduces formal verification and finally AMLD machinery when the security problem motivates it. The manuscript follows a defensive theorem policy: almost all formal results are meant to show how a construction reduces, contains, detects, or repairs a stated vulnerability class under explicit assumptions. The draft is for review; citations, source anchors, and implementation references should be frozen to a specific GitHub commit before publication.

Reader's Guide

This book is organized so that a reader does not need to know AMLD in advance. The first half can be read as a compact mathematical introduction to security architecture. AMLD appears only after the reader has encountered several recurring needs: keeping trusted and speculative information separate, distinguishing several kinds of formal settlement, remembering verified successes and failures, and testing new reasoning systems behind a gate before allowing them to affect trusted state.

The writing rule is simple: *motivation first, formalism immediately after*. “Immediately” describes the order, not the length. Each section should explain the security problem in ordinary language, introduce only the notation needed to represent it, state a defensive result where one is justified, prove it, and then say what the result does not establish.

Defensive theorem policy

The preferred form of a result is

defense D reduces or contains vulnerability class X

under a declared model and assumptions. A second preferred form compares two defensive designs,

$$D_2 \succ_X D_1,$$

meaning that, for the metric specified in the theorem, D_2 offers strictly stronger protection against X than D_1 . Results about finding defects are included when they support auditing or repair, but the manuscript is not organized as a manual for discovering or exploiting vulnerabilities.

Three recurring boundaries

First, no finite benchmark proves universal security. Second, a verifier is only as meaningful as the specification and environment it checks. Third, a result that improves confidentiality, integrity, or compromise containment can still worsen availability, usability, cost, or another security coordinate. Security is therefore treated as a vector of properties rather than a single adjective.

Contents

Reader's Guide	i
I Foundations of Security	1
1 Information	2
1.1 Bits, symbols, strings, and states	2
1.2 Information available to an observer	2
1.3 Probability and uncertainty	2
1.4 Entropy and concentration	3
1.5 Side information and post-processing	3
1.6 Information cannot arise from nowhere	3
2 Secrets	5
2.1 What is a secret?	5
2.2 Secret spaces and distributions	5
2.3 Guessing probability	5
2.4 Random and human-selected secrets	6
2.5 Independence and correlation	6
2.6 Reuse, compromise, and replacement	6
3 What Security Means	7
3.1 Assets and undesirable events	7
3.2 Confidentiality	7
3.3 Integrity	7

3.4	Availability	7
3.5	Authenticity, privacy, and accountability	8
3.6	Security as a vector	8
3.7	Tradeoffs and Pareto improvement	8
4	Separation and Isolation	9
4.1	Security domains	9
4.2	Separation of data	9
4.3	Separation of credentials	10
4.4	Separation of authority	10
4.5	Least privilege	10
4.6	Blast radius	10
4.7	Narrow bridges between domains	11
5	Identity, Authentication, and Authorization	12
5.1	Identity	12
5.2	Claims about identity	12
5.3	Authentication	12
5.4	Authorization	12
5.5	Sessions	13
5.6	Revocation and continuing authorization	13
6	Credentials and Factors	14
6.1	Knowledge factors	14
6.2	Possession factors	14
6.3	Biometric factors	14
6.4	Recovery credentials	14
6.5	Multiple factors	15
6.6	Independence between factors	15
6.7	Combining factors safely	15
7	Threat Models	16
7.1	What is being protected?	16

7.2	What does the adversary know?	16
7.3	What can the adversary observe?	16
7.4	What can the adversary modify?	16
7.5	Resource bounds	17
7.6	Trusted components	17
7.7	Security claims are relative to a model	17
II	Authentication and Guarded Verification	18
8	The Security Verifier	19
8.1	Inputs and decisions	19
8.2	Stateless verification	19
8.3	Stateful verification	19
8.4	Retry counters and timers	19
8.5	Risk and account state	20
8.6	ACCEPT, REJECT, DELAY, and BLOCKED	20
8.7	Verifying the verifier	20
9	Routes and Attack Surfaces	21
9.1	Primary routes	21
9.2	Recovery routes	21
9.3	Enrollment and reset	21
9.4	Remembered sessions	21
9.5	Alternate interfaces	21
9.6	Route interaction	22
9.7	The weakest sufficient route	22
10	Online and Offline Security	23
10.1	Interactive verification	23
10.2	Attempt budgets	23
10.3	Rate limiting	23
10.4	Delay and lockout	23

10.5	Offline verification	23
10.6	Salts and instance separation	24
10.7	Why defenses do not transfer automatically	24
11	Probability of Compromise	25
11.1	Security distributions	25
11.2	Candidate ordering	25
11.3	Top- N probability mass	25
11.4	Min-entropy	26
11.5	Conditional information	26
11.6	Compromise bounds	26
11.7	Comparing defenses quantitatively	26
III	Containment and Defensive Architecture	27
12	Containment	28
12.1	Prevention versus containment	28
12.2	Local failure versus system failure	28
12.3	Compromise propagation	28
12.4	Privilege boundaries	28
12.5	Damage bounds	29
12.6	Recovery after containment	29
13	Controlled Interfaces	30
13.1	Why systems need bridges	30
13.2	Minimal interfaces	30
13.3	Typed inputs and outputs	30
13.4	Input validation	30
13.5	Authority carried across an interface	30
13.6	Interface invariants	31
13.7	Verified boundary transitions	31

14 The Ocean as a Security Moat	32
14.1 The Ocean benchmark as an adversarial environment	32
14.2 Candidate systems outside the trusted perimeter	32
14.3 Security invariants tested in the Ocean	32
14.4 Verifier-gated admission	32
14.5 Failure becomes a permanent regression case	33
14.6 Holdouts and unseen Ocean cases	33
14.7 What passing the Ocean does and does not prove	33
 IV Formal Security	 34
15 Specifications and Invariants	35
15.1 Intended behavior	35
15.2 Implemented behavior	35
15.3 Safety properties	35
15.4 Invariants	35
15.5 Counterexamples	36
15.6 Partial and complete specifications	36
 16 Security as Conformance	 37
16.1 Required acceptance	37
16.2 Implemented acceptance	37
16.3 Acceptance-soundness	37
16.4 False acceptance	37
16.5 False rejection	38
16.6 Full conformance	38
 17 Defensive Theorems	 39
17.1 Hardening theorems	39
17.2 Isolation theorems	39
17.3 Containment theorems	39
17.4 Rate-control theorems	39

17.5	Factor-composition theorems	39
17.6	Recovery-route theorems	40
17.7	Repair-preservation theorems	40
V	AMLD Security	41
18	Security as a Settlement Problem	42
18.1	Why binary prove-or-fail is too crude	42
18.2	P: proving the security property	42
18.3	R: producing a verified counterexample	42
18.4	I: independence	42
18.5	C: inconsistency	43
18.6	UNKNOWN	43
18.7	Independent verification	43
19	BANK and FUTUREBANK	44
19.1	The security problem that motivates two stores	44
19.2	The BANK	44
19.3	The FUTUREBANK	44
19.4	Quarantine before verification	44
19.5	Promotion into the BANK	45
19.6	Preventing speculative contamination	45
19.7	Why BANK/FUTUREBANK precedes P/R/I/C conceptually	45
20	Security Memory	46
20.1	Remembering successful defenses	46
20.2	Remembering failed defenses	46
20.3	Rejected defect hypotheses	46
20.4	Negative knowledge	46
20.5	Regression memory	46
20.6	Cross-system transfer	47
20.7	Append-only security history	47

21 AMLD Control and Repair	48
21.1 Security objectives	48
21.2 Independent control variables	48
21.3 Dissatisfaction with a defense	48
21.4 Ordinary optimization	48
21.5 Structural revision	49
21.6 Comparing candidate repairs	49
21.7 Verifier-gated deployment	49
 VI Experiments and Research Frontier	 50
22 Security Experiments	51
22.1 What constitutes an experiment?	51
22.2 Threat-model declaration	51
22.3 LAB, paired, holdout, and target cases	51
22.4 Security baselines	51
22.5 Ablations	51
22.6 Preregistration	52
22.7 Reproducibility and certificates	52
 23 Measuring Defensive Improvement	 53
23.1 Compromise probability	53
23.2 Blast radius	53
23.3 Availability cost	53
23.4 Verification cost	53
23.5 Repair cost	53
23.6 Multi-objective security	54
23.7 Pareto frontiers	54
 24 AMLD Security Conjectures	 55
24.1 Audit-over-secret advantage	55
24.2 Cross-route shared-BANK advantage	55

24.3 Adaptive flood-control frontier	55
24.4 Verifier-history repair advantage	55
24.5 Ocean-regression-moat advantage	56
24.6 Separation-aware AMLD control	56
24.7 Persistent-failure-memory advantage	56
25 Claims We Should Not Make	57
25.1 No finite test proves universal security	57
25.2 More searching is not necessarily safer	57
25.3 More reflection is not necessarily safer	57
25.4 More isolation is not automatically better	57
25.5 Verification is only as meaningful as its specification	58
25.6 A secure component does not imply a secure composition	58
25.7 UNKNOWN is not failure	58
VII Back Matter	59
A Glossary of Terms	60
B Glossary of Symbols and Notation	62
C Draft Source and GitHub Reference Map	65
D References	66
Draft Status	68

Part I

Foundations of Security

Chapter 1

Information

1.1 Bits, symbols, strings, and states

Security begins with distinctions among possible states. A bit distinguishes two states; a finite string distinguishes among finitely many strings; a database record, session object, or machine configuration distinguishes among a larger collection of structured states. The mathematics does not require every security object to be represented literally as a bit string, but finite encodings are useful because they make observation and uncertainty explicit.

Let S be a state space and let X be an S -valued random variable. An observer does not necessarily see X directly. Instead the observer sees another variable Z whose value is produced by the system, the environment, or prior knowledge. Security questions often ask how much Z permits the observer to infer about X .

1.2 Information available to an observer

The phrase “available information” should always be relative to an observer and a model. A server process may see a secret key that a web client does not; an administrator may see account state that an unauthenticated requester does not; an offline auditor may possess a system snapshot that a live user does not. There is no observer-independent list of “the information.”

If Z is the information available to an observer, every security statement about inference from Z should be read as conditional on the declared model that determines Z . Additional legitimate side information is represented by enlarging Z , not by silently changing the probability distribution later.

1.3 Probability and uncertainty

For an event E , $\Pr(E)$ denotes its probability. For random variables X and Z , the conditional probability

$$\Pr(X = x \mid Z = z)$$

models the observer’s uncertainty about X after learning $Z = z$. In a security model, the conditional distribution is often more important than a nominal keyspace or password alphabet. A space may be huge while the actual distribution is concentrated on a small number of likely values.

1.4 Entropy and concentration

Entropy is one way to summarize uncertainty, but it is not the only one and it is not always the quantity that controls a security decision. For bounded guessing, the largest probabilities and the mass of the most likely candidates can matter more directly. We will use Shannon entropy when average information is relevant, min-entropy when maximum point probability is relevant, and top- N mass when a verifier processes only a bounded number of candidates.

1.5 Side information and post-processing

Suppose $Y = f(Z)$ is a deterministic summary of information Z . Whatever an observer can do after seeing Y could also have been done after seeing the richer value Z by first computing $f(Z)$. This elementary observation gives a useful defensive principle.

Proposition 1.1 (Coarsening cannot improve optimal one-shot guessing). *Let X, Z be finite random variables and let $Y = f(Z)$. Let*

$$p^*(X \mid Z) = \sum_z \Pr(Z = z) \max_x \Pr(X = x \mid Z = z)$$

be the optimal one-shot guessing probability after observing Z , and define $p^(X \mid Y)$ analogously. Then*

$$p^*(X \mid Y) \leq p^*(X \mid Z).$$

Proof. Any decision rule $g(Y)$ is also a decision rule based on Z , namely $g(f(Z))$. The class of strategies available after observing Y is therefore a subset of the class available after observing Z . Maximizing over the larger class cannot give a smaller success probability. $\square$

Security interpretation. If a system can expose a coarser view without breaking legitimate functionality, the coarser view cannot by itself make optimal inference about a protected variable easier than the richer view it replaces. This is not a claim that every redaction is sufficient; it says only that removing observable distinctions cannot increase the information available through that channel.

1.6 Information cannot arise from nowhere

A search procedure can reorder, combine, and test hypotheses, but it cannot create statistical dependence between an unknown secret and observations that are independent of that secret. This simple fact will later limit what an ATP, AMLD controller, or any other search system can do in a

password or hidden-state problem. Imagination can propose structure; it cannot manufacture secret bits that are absent from the observations and model.

Chapter 2

Secrets

2.1 What is a secret?

A secret is not merely a string marked “secret.” It is a value whose disclosure is restricted relative to some observer or class of observers. Formally, let W be a random variable taking values in a finite or countable set $\mathcal{S}$. The security model declares which observations Z are available to an observer and which events count as unauthorized recovery or use of W .

2.2 Secret spaces and distributions

The set $\mathcal{S}$ describes what values are possible. The distribution $\pi(w) = \Pr(W = w)$ describes how those values are selected. These are different objects. A large set with a sharply concentrated distribution may provide less practical resistance to guessing than a smaller set with a nearly uniform distribution.

2.3 Guessing probability

Order the secret probabilities as

$$\pi_{(1)} \geq \pi_{(2)} \geq \cdots.$$

The best one-shot guessing probability is $\pi_{(1)}$. With at most N distinct guesses and no additional information from failures, the best achievable success probability is the top- N mass

$$G_N(\pi) = \sum_{i=1}^N \pi_{(i)},$$

capped at 1 if the support has fewer than N elements.

2.4 Random and human-selected secrets

A cryptographically suitable generator can make a secret distribution close to uniform over a declared support. Human choice often introduces concentration, reuse, and correlation with other information. The mathematical point is not that one population behaves in a single way; it is that a security theorem should use the actual modeled distribution rather than infer strength from formatting rules alone.

2.5 Independence and correlation

Suppose two domains use secrets W_A and W_B . If compromise of W_A reveals information about W_B , the second domain inherits some of the first domain's failure. Independent generation is one way to block that propagation channel.

Proposition 2.1 (Independent-secret containment). *If W_A and W_B are independent and an event C_A depends only on W_A and variables independent of W_B , then*

$$\Pr(W_B = w \mid C_A) = \Pr(W_B = w)$$

whenever the conditional probability is defined.

Proof. By the stated independence, C_A is independent of W_B . Conditioning on an independent event leaves the distribution of W_B unchanged. $\square$

Security interpretation. Independent credentials do not make domain B invulnerable. They remove one specific propagation route: learning or compromising an A -only secret does not, by that fact alone, update the distribution of B 's independent secret.

2.6 Reuse, compromise, and replacement

Reuse intentionally destroys some independence. When the same credential authorizes several systems, compromise of that credential may expand the blast radius across every system that accepts it. Rotation or replacement helps only if the replacement is generated safely and the old credential is actually revoked wherever it had authority.

Chapter 3

What Security Means

3.1 Assets and undesirable events

A security model should identify assets and events that matter. An asset may be data, computational authority, money, availability, reputation, a signing key, a physical process, or the correctness of a formal knowledge base. An undesirable event may be disclosure, unauthorized modification, denial of service, impersonation, or propagation of a local compromise.

3.2 Confidentiality

Confidentiality limits what unauthorized observers can learn. It is therefore naturally connected to the information model of Chapter 1. A confidentiality result should identify the protected variable, the observer, the allowed observations, and the inference or disclosure event being bounded.

3.3 Integrity

Integrity concerns unauthorized or unintended changes. It can be stated as an invariant over states, a restriction on transitions, or a conformance relation between implemented behavior and required behavior. Later chapters will treat integrity as one of the clearest places where verification and AMLD-style settlement can contribute.

3.4 Availability

Availability asks whether legitimate users or components can obtain service when required. A control that decreases unauthorized acceptance can still reduce availability. Hard account lockout is the canonical example: fewer online guesses may be processed, but an outsider may also be able to consume another user's failure budget if the policy is badly designed.

3.5 Authenticity, privacy, and accountability

Authenticity concerns whether an object, message, or action can be tied to its claimed source under the model. Privacy is broader than secrecy and may include limiting linkability, inference, or collection. Accountability concerns whether actions can be attributed and reviewed. These properties can support one another, but they are not synonyms.

3.6 Security as a vector

Let a design D have a security-and-cost vector

$$S(D) = (C, I, A, P, B, K, \dots),$$

where coordinates may represent confidentiality, integrity, availability, privacy, blast radius, operational cost, or other declared quantities. The exact coordinates depend on the model. A claim that D_2 is “more secure” than D_1 is incomplete unless the comparison relation on these vectors is specified.

3.7 Tradeoffs and Pareto improvement

A particularly clean result is a Pareto improvement: one design improves at least one declared coordinate without worsening any others. Many real defenses are not Pareto improvements; they trade availability, cost, latency, or usability for reduced compromise probability. The book will report those tradeoffs rather than hide them behind a single score.

Chapter 4

Separation and Isolation

4.1 Security domains

A security domain is a collection of state and authority intended to be governed by a common policy boundary. Separation asks which information, credentials, permissions, and failure consequences should remain local to one domain instead of being shared by default.

Figure asset not present in this source-only copy. See the compiled PDF or the source bundle.

Figure 4.1: A motivating schematic with parallel A and B database, user, and critic structures. The drawing is not itself a security architecture; it illustrates the question that motivates one: which identities, data, and authorities should remain separate, and which bridges are actually necessary?

Figure 4.1 is deliberately ordinary. The security issue appears as soon as two sides that perform related work are represented separately. If one side is compromised, should the other side automatically become readable, writable, or administratively controllable? If the answer is no, the architecture needs a boundary that makes “no” true in implementation, not merely in documentation.

4.2 Separation of data

Let D_A and D_B be two data resources. A minimal separation goal is that authority to read or write one does not automatically imply authority over the other. The implication

$$\text{Read}_A(u) \Rightarrow \text{Read}_B(u)$$

should fail unless the policy explicitly requires it. The same applies to write authority.

4.3 Separation of credentials

A credential that proves authority in domain A should not silently function as a universal credential for B . This does not forbid federation or single sign-on; it requires any authority transfer to be explicit, scoped, and governed by a bridge whose behavior can be reviewed.

4.4 Separation of authority

Data can be stored in separate databases while applications still run under one highly privileged account. In that case physical or logical data separation may provide less containment than expected. Let $\mathcal{A}_A$ and $\mathcal{A}_B$ denote the actions authorized in the two domains. Stronger separation restricts the intersection and the conversion paths between these authority sets.

4.5 Least privilege

Least privilege gives a component only the authority needed for its assigned function and duration. The principle is defensive because it reduces the set of actions available after the component itself is compromised. It is a statement about consequence containment, not a guarantee that compromise will not happen.

4.6 Blast radius

Let a directed graph $G = (V, E)$ represent possible compromise propagation under the declared threat model. For a component $c \in V$, define the blast radius

$$B_G(c) = \text{Reach}_G(c),$$

the set of protected vertices reachable from c along permitted or exploitable propagation edges represented in the model.

Theorem 4.1 (Edge-removal containment). *Let $G' = (V, E')$ with $E' \subseteq E$. Then for every $c \in V$,*

$$B_{G'}(c) \subseteq B_G(c).$$

If at least one vertex reachable from c in G becomes unreachable in G' , then the inclusion is strict.

Proof. Every path in G' uses only edges in E' , hence only edges in E , and is therefore also a path in G . Removing edges cannot create a new path. Strictness follows when a previously reachable vertex has no remaining path. $\square$

Security interpretation. A correctly implemented isolation boundary can improve security even if it does not reduce the probability that c is compromised. It can reduce what follows from that compromise.

4.7 Narrow bridges between domains

Absolute isolation is often incompatible with useful systems. The defensive goal is therefore not “no bridge” but “the smallest bridge that supports the legitimate function.” If an object x crosses from domain A to domain B , let $P(x)$ be the predicate stating that x satisfies the boundary policy. Later we will require every state-changing crossing to pass a gate that establishes $P(x)$ before protected state is modified.

Chapter 5

Identity, Authentication, and Authorization

5.1 Identity

Identity is the entity or account to which claims and permissions are attached. A username may name an identity, but a name alone does not authenticate the claimant.

5.2 Claims about identity

A claimant presents evidence associated with an identity. The evidence may be a password, cryptographic response, device-bound credential, biometric comparison, recovery artifact, or another approved mechanism. The system must distinguish the claim from the evidence supporting it.

5.3 Authentication

Authentication is the process that evaluates whether presented evidence satisfies the system's authentication policy. It establishes a claim to the degree and scope specified by that policy; it does not by itself answer what actions should be permitted afterward.

5.4 Authorization

Authorization maps authenticated or otherwise established context to permitted actions. Formally, if x is system state and a an action, a policy may define

$$\text{Allowed}(u, a, x) \in \{0, 1\}.$$

Security requires the implementation's authorization decision to conform to the required policy over the relevant state space.

5.5 Sessions

A session carries authentication state forward in time. Session security therefore becomes a problem of protecting a capability-like object: creation, binding, storage, expiration, renewal, and revocation all affect whether past authentication continues to justify present authority.

5.6 Revocation and continuing authorization

A credential or session that was once valid may cease to be valid. State transitions such as account disablement, role change, key rotation, or recovery should therefore be included in the authorization model rather than treated as administrative details outside the proof.

Chapter 6

Credentials and Factors

6.1 Knowledge factors

Knowledge factors rely on information the claimant is expected to know. Their security depends on the distribution of the secret, protection of verifier data, rate controls where applicable, and the absence of alternate routes that bypass the intended requirement.

6.2 Possession factors

Possession factors rely on control of a device, key, token, or other object. The relevant threat model concerns theft, copying, remote use, enrollment, and recovery. “Possession” is therefore a policy claim about control, not a metaphysical property of the object.

6.3 Biometric factors

Biometric systems compare measured features with stored or derived templates. They differ from ordinary secrets because a person cannot rotate a fingerprint or face in the same way a password can be changed. Error rates, spoof resistance, privacy of templates, and fallback routes all belong in the model.

6.4 Recovery credentials

Recovery exists because primary credentials can be lost. That makes recovery a security route, not merely a support function. If recovery is sufficient to regain the same authority as the primary route, the whole account cannot be modeled accurately by analyzing the primary route alone.

6.5 Multiple factors

A simple conjunction model accepts only when several predicates hold:

$$\text{Accept}(z) = \text{Pwd}(z) \wedge \text{Factor}(z) \wedge \text{Eligible}(z) \wedge \text{Policy}(z).$$

The actual implementation may be more stateful, but the conjunction is useful for one basic containment fact.

Proposition 6.1 (Conjunctive-factor containment). *If acceptance requires predicates $P_1, \dots, P_k$ all to hold, then the accepted event set is a subset of the event set satisfying any individual factor P_i .*

Proof. Every event satisfying $P_1 \wedge \dots \wedge P_k$ satisfies each conjunct P_i . □

6.6 Independence between factors

Two factors provide less separation if the same compromise mechanism yields both. Factor independence is therefore a property of the threat model, not merely of labels such as “something you know” and “something you have.”

6.7 Combining factors safely

A multi-factor design can be weakened by enrollment, recovery, remembered sessions, fallback logic, or inconsistent enforcement. A defensive proof must therefore refer to the complete acceptance predicate and state machine rather than assume that the presence of two factor names establishes two-factor security.

Chapter 7

Threat Models

7.1 What is being protected?

A threat model starts with assets and security properties. Without a protected object and an undesirable event, it is impossible to say what a defense is supposed to reduce.

7.2 What does the adversary know?

Knowledge is modeled as side information Z . Public documentation, disclosed identifiers, prior breaches, observable errors, and legitimate protocol messages may all contribute. The model should not grant secret information silently, and it should not deny information that the environment necessarily exposes.

7.3 What can the adversary observe?

Observation includes outputs, timing, state changes visible through interfaces, and any other declared channel. Defensive design often reduces unnecessary distinctions in these observations.

7.4 What can the adversary modify?

Modification authority can include requests, local files, network messages, configuration inputs, or physical access depending on the model. A theorem that assumes read-only observation does not automatically survive a model that permits state modification.

7.5 Resource bounds

Security claims may depend on time, number of queries, memory, cost, or computational assumptions. The relevant bound should appear in the statement rather than be left as an implicit word such as “practical.”

7.6 Trusted components

Every proof has a trusted base: hardware, operating system, parser, compiler, cryptographic primitive, verifier, policy specification, or some subset of these. Formal work improves clarity by naming that base rather than pretending it is empty.

7.7 Security claims are relative to a model

Boundary note 7.1. A proof of security property P in model M establishes P in M . It does not, by syntax alone, establish P in a stronger threat model M' with additional adversary capabilities or different environmental assumptions.

This is not a weakness of proof. It is the reason assumptions must be explicit.

Part II

Authentication and Guarded Verification

Chapter 8

The Security Verifier

8.1 Inputs and decisions

A verifier maps an input and relevant state to an outcome. For a guarded authentication process, a useful abstract form is

$$\text{Auth}_t(g, x_t) \in \{\text{ACCEPT}, \text{REJECT}, \text{DELAY}, \text{BLOCKED}\}.$$

Here g is the submitted candidate or credential bundle and x_t is system state at time t .

8.2 Stateless verification

A stateless verifier's answer depends only on the current input and a fixed environment. Mathematical proof checking is often modeled this way: once a certificate is valid relative to a fixed environment version, the history of previous failed search attempts does not make the certificate invalid.

8.3 Stateful verification

Live security verifiers are commonly stateful. A correct credential can be refused because an account is disabled, a factor is pending, a risk policy blocks the request, or a retry budget has been exhausted. Security formalization therefore needs transition semantics as well as a credential predicate.

8.4 Retry counters and timers

Let n_t be a failure counter and τ_t a timer or delay state. A transition rule updates these values after each outcome. A claimed attempt bound N is meaningful only if the modeled transition system actually prevents more than N processed attempts within the declared episode or horizon.

8.5 Risk and account state

Risk scores and account status can change the acceptance predicate. Adaptive policies can improve some tradeoffs but also complicate verification because the verifier's behavior must now be analyzed over a larger state space.

8.6 ACCEPT, REJECT, DELAY, and BLOCKED

The four outcomes separate semantic rejection from temporary delay and policy blocking. This avoids pretending that every non-acceptance is the same event. Availability analysis especially depends on the distinction.

8.7 Verifying the verifier

The deeper target is not to guess what the verifier will do but to prove that what it does conforms to an intended specification. Let $\text{Impl}(z)$ denote implemented acceptance for a complete event z and $\text{Req}(z)$ the required acceptance predicate. The one-sided safety claim is

$$c_{\text{auth}} : \quad \forall z [\text{Impl}(z) \Rightarrow \text{Req}(z)].$$

A counterexample z^* with $\text{Impl}(z^*)$ true and $\text{Req}(z^*)$ false is a certificate of an unintended acceptance defect in the modeled system.

Chapter 9

Routes and Attack Surfaces

9.1 Primary routes

A system may have a primary login flow, but overall security depends on every sufficient route to the protected authority. Modeling only the most visible route can overstate the effect of strengthening it.

9.2 Recovery routes

Recovery may restore credentials, issue a session, or otherwise regain access. If it can reach the same protected state, it belongs in the same end-to-end security analysis.

9.3 Enrollment and reset

Enrollment creates authority; reset reassigns or replaces it. These transitions can be more consequential than ordinary login because they change which credentials will be accepted in the future.

9.4 Remembered sessions

A remembered session can act as an alternate route that bypasses fresh primary authentication. Its theft, expiration, revocation, and binding conditions therefore affect system-level security.

9.5 Alternate interfaces

Web, mobile, API, administrative, and legacy interfaces may enforce different controls. A policy theorem about one interface does not automatically cover another.

9.6 Route interaction

Defects can arise only when individually reasonable routes interact: a recovery flow may fail to invalidate sessions, or a role change may not propagate to a separate service. Shared state and transitions should therefore be explicit in the model.

9.7 The weakest sufficient route

Let routes $R_1, \dots, R_k$ each be sufficient to obtain protected authority, and let C_i denote a modeled cost of successfully traversing route R_i . An actor able to choose the least-cost sufficient route faces system-level cost at most

$$C_{\text{system}} \leq \min_i C_i.$$

This is an upper bound, not a complete metric. Its defensive lesson is that strengthening one route while leaving a cheaper sufficient route unchanged cannot force the overall minimum above that unchanged route.

Chapter 10

Online and Offline Security

10.1 Interactive verification

An online attempt is mediated by a live system that can update state, rate-limit, delay, block, or require additional factors. The number and timing of processed attempts are therefore part of the security mechanism.

10.2 Attempt budgets

If an episode permits at most N processed guesses and failed guesses reveal no additional secret-dependent information beyond rejection, the direct guessing success probability is bounded by the top- N mass $G_N(\pi)$.

10.3 Rate limiting

Rate limiting reduces how quickly attempts can be processed. It may create an effective attempt budget over a finite horizon even when no permanent lockout occurs.

10.4 Delay and lockout

Delay and lockout can reduce online guessing opportunity, but aggressive policies may also reduce legitimate availability. The correct comparison is therefore multi-objective.

10.5 Offline verification

If an attacker or auditor possesses data sufficient to test candidates locally, the live server's retry counter no longer directly limits those local checks. The relevant defense becomes the cost and

structure of the offline verifier data and the underlying secret distribution, together with the broader threat model.

10.6 Salts and instance separation

Unique salts separate precomputation across verifier instances. A table computed for one salt does not directly provide outputs for a different salt unless additional computation or information supplies those outputs. Salting does not make weak secrets strong; it blocks a particular reuse of precomputed work.

10.7 Why defenses do not transfer automatically

A defense theorem should name the verification setting. Online throttling, offline password hashing cost, factor requirements, and hardware-backed key protection address different mechanisms. “Secure against guessing” is too broad unless the mode of verification is declared.

Chapter 11

Probability of Compromise

11.1 Security distributions

Let π be the conditional distribution of a protected secret or event given the side information available in the modeled episode. All probability statements in this chapter are relative to that conditioning.

11.2 Candidate ordering

An optimal bounded guessing strategy tries candidates in nonincreasing probability order when guesses are independent tests and failure yields no additional information. This converts the distribution into an ordered list $\pi_{(1)}, \pi_{(2)}, \dots$

11.3 Top- N probability mass

Define

$$G_N(\pi) = \sum_{i=1}^N \pi_{(i)}.$$

It measures how much probability mass is exposed to the best N guesses under the stated model.

Theorem 11.1 (Finite-attempt bound). *If at most N distinct candidates can be tested, each test only reports whether that candidate is correct, and the prior distribution remains the relevant model, then no strategy can exceed success probability $G_N(\pi)$.*

Proof. Any set of at most N tested candidates has total prior mass no greater than the sum of the N largest point probabilities. Testing those N largest candidates attains the bound. $\square$

11.4 Min-entropy

If the conditional min-entropy is

$$H_\infty(W \mid Z = z) = -\log_2 \max_w \Pr(W = w \mid Z = z),$$

then every point probability is at most 2^{-H_∞} . Consequently,

$$G_N(\pi) \leq \min\{1, N2^{-H_\infty}\}.$$

11.5 Conditional information

The distribution should already incorporate legitimate side information. Adding side information can concentrate or sometimes redistribute the conditional probabilities. Security comparisons must therefore hold the information model fixed or state exactly how it changes.

11.6 Compromise bounds

A defense can reduce modeled compromise probability in several different ways: reduce the number N of opportunities, flatten the relevant distribution, require independent additional predicates, or contain what authority follows from success.

11.7 Comparing defenses quantitatively

When two defenses affect different mechanisms, a scalar comparison may be misleading. One may lower online success probability but increase denial risk; another may leave primary compromise probability unchanged while sharply reducing blast radius. Chapter 3 therefore treats security improvement as vector-valued.

Part III

Containment and Defensive Architecture

Chapter 12

Containment

12.1 Prevention versus containment

Prevention reduces the probability that an undesirable event begins. Containment reduces the consequences after it begins. Mature security usually requires both because no realistic system can assume every preventive control will succeed forever.

12.2 Local failure versus system failure

A local component failure should not automatically become a system-wide failure. This is the architectural form of the separation principle: the system should preserve important invariants outside the failed domain whenever the required function permits it.

12.3 Compromise propagation

Propagation can be modeled by reachability, capabilities, shared credentials, shared storage, or state-transition dependencies. The appropriate model depends on what “compromise” means in the system under study.

12.4 Privilege boundaries

A privilege boundary is useful only if it changes the actions available after crossing or failing to cross it. Labels, process names, or separate databases do not provide containment unless the implementation enforces different authority.

12.5 Damage bounds

Let $L(v)$ denote loss associated with compromise of protected vertex v and $B(c)$ the blast radius from c . A simple modeled worst-case loss is

$$L_{\max}(c) = \sum_{v \in B(c)} L(v).$$

Reducing $B(c)$ can reduce this upper bound even when the probability of compromising c is unchanged.

12.6 Recovery after containment

Containment should support recovery: revoke compromised credentials, reconstruct trusted state, verify repaired configurations, and restore service without silently reintroducing the same propagation path.

Chapter 13

Controlled Interfaces

13.1 Why systems need bridges

Useful systems communicate. The goal of isolation is therefore controlled interaction, not universal disconnection. Every bridge should justify the data and authority it carries.

13.2 Minimal interfaces

A minimal interface exposes only operations and data needed for the legitimate function. Fewer operations can reduce the set of states reachable from untrusted input, although minimality alone does not prove correctness.

13.3 Typed inputs and outputs

Types, schemas, and protocol grammars constrain what a receiver is expected to process. They are useful as formal boundaries when the parser and implementation are included in the trusted model.

13.4 Input validation

Validation checks whether an input satisfies the preconditions for a transition. Defensive modeling treats validation as a predicate with defined semantics, not as a vague instruction to “sanitize input.”

13.5 Authority carried across an interface

Some messages are pure data; others carry authority. A signed request, session token, capability, or administrative command can change what the receiver is permitted to do. Interfaces should make such authority explicit and scope it narrowly.

13.6 Interface invariants

Let $I(k)$ be an invariant of protected state k , and let $P(x)$ be the gate predicate for an incoming object. A safe transition rule T should satisfy

$$I(k) \wedge P(x) \Rightarrow I(T(k, x)).$$

13.7 Verified boundary transitions

Theorem 13.1 (Gate-preservation principle). *Suppose protected state begins in k_0 with $I(k_0)$. Every state-changing external input x_t is admitted only when a gate establishes $P(x_t)$, and every admitted transition preserves I as above. Then every protected state reachable through the modeled interface satisfies I .*

Proof. Induct on the number of admitted transitions. The base state satisfies I . If $I(k_t)$ holds and the gate admits x_t , then $P(x_t)$ holds; preservation gives $I(k_{t+1})$. Rejected inputs cause no modeled state-changing transition. $\square$

Boundary note 13.2. The theorem depends on the claim that every relevant state-changing path goes through the modeled gate. A hidden administrative channel, shared memory path, or unmodeled privilege can invalidate the premise.

Chapter 14

The Ocean as a Security Moat

14.1 The Ocean benchmark as an adversarial environment

The Ocean in this manuscript is the ATP test environment, repurposed as a defensive admission test. A candidate ATP, controller, memory update, or architectural revision is exercised on a deliberately varied suite before it is trusted with deployment authority. The Ocean is not itself proof of security; it is a place where failures can be exposed without granting the candidate direct control over the protected core.

14.2 Candidate systems outside the trusted perimeter

A candidate version should be treated as untrusted until it passes the required gate. This is ordinary separation applied to the development process: experimental capability and trusted operational authority are different domains.

14.3 Security invariants tested in the Ocean

Relevant invariants may include: only verifier-accepted mathematical objects enter the trusted BANK; speculative states cannot overwrite trusted memory; UNKNOWN is not silently converted to success; provenance is preserved; resource limits are respected; and a failed verifier cannot be bypassed by another component.

14.4 Verifier-gated admission

Let A be a candidate system and let $\mathcal{O}_t = \{o_1, \dots, o_n\}$ be the required Ocean suite at time t . Let $V_I(A, o)$ be an independent check that candidate A preserved security invariant set I during trial o . Define

$$\text{Pass}_{\mathcal{O}_t}(A) = 1 \iff V_I(A, o) = 1 \text{ for every required } o \in \mathcal{O}_t.$$

Deployment policy may require this predicate without claiming it is sufficient for universal security.

14.5 Failure becomes a permanent regression case

When a verified failure case r_t is accepted as a useful regression test, update

$$\mathcal{O}_{t+1} = \mathcal{O}_t \cup \{r_t\}.$$

The intended moat therefore accumulates evidence from history instead of forgetting failures after they are repaired.

Proposition 14.1 (Tested-violation exclusion). *Suppose deployment requires $\text{Pass}_{\mathcal{O}_t}(A) = 1$. Then no candidate that exhibits a verifier-recognized violation of I on any required Ocean case in $\mathcal{O}_t$ is admitted by that policy.*

Proof. Such a violation makes $V_I(A, o) = 0$ for at least one required o , hence $\text{Pass}_{\mathcal{O}_t}(A) = 0$ by definition. $\square$

14.6 Holdouts and unseen Ocean cases

If every test becomes visible during development, a candidate can overfit the known suite. Holdout families, newly generated cases, and versioned test partitions are therefore useful for estimating generalization. They still do not prove universal security.

14.7 What passing the Ocean does and does not prove

An Ocean pass proves only what the gate and suite are specified to establish. It can exclude candidates with known tested failures; it cannot establish behavior on every future environment, every implementation bug, or every threat model. The moat is strongest when it is combined with independent verification, separation of authority, and a permanent regression history.

Part IV

Formal Security

Chapter 15

Specifications and Invariants

15.1 Intended behavior

A specification states what behavior is required, permitted, or forbidden. It may be partial: a safety specification can forbid unauthorized acceptance without defining every user-interface detail.

15.2 Implemented behavior

The implementation induces its own transition and acceptance semantics. Formal assurance compares these semantics with the specification rather than assuming the implementation inherits the specification by intention.

15.3 Safety properties

A safety property says, informally, that a bad event does not occur. Many security properties take this form: no unauthorized write, no acceptance without required factors, no unverified BANK deposit.

15.4 Invariants

An invariant I is a property intended to hold in every reachable state of the modeled system. Inductive invariants are especially useful because they can be proved from an initial condition and transition-preservation rule.

15.5 Counterexamples

A counterexample is a concrete object or trace showing that a universal claim fails. In a defensive workflow, a checked counterexample is valuable because it identifies a specification-conformance defect that can be repaired and later added to regression testing.

15.6 Partial and complete specifications

No specification is automatically complete. A proof may establish conformance to a stated policy while the policy itself omits an important security requirement. The book will therefore distinguish implementation conformance from adequacy of the specification.

Chapter 16

Security as Conformance

16.1 Required acceptance

Let $\text{Req}(z)$ be the predicate stating that complete event z is intended to be accepted under the security policy.

16.2 Implemented acceptance

Let $\text{Impl}(z)$ state that the modeled implementation accepts z . The two predicates may differ because of software defects, inconsistent state handling, incomplete policy enforcement, or a deliberately partial model.

16.3 Acceptance-soundness

The one-sided security claim

$$\forall z [\text{Impl}(z) \Rightarrow \text{Req}(z)]$$

states that every implemented acceptance is authorized by the specification. It does not require the implementation to accept every event that the specification permits.

Proposition 16.1 (Counterexample sufficiency). *If there exists z^* such that $\text{Impl}(z^*)$ and $\neg \text{Req}(z^*)$, then acceptance-soundness is false.*

Proof. The witness z^* falsifies the universal implication. □

16.4 False acceptance

A false acceptance is an event accepted by the implementation but forbidden by the specification. It is a safety defect relative to the declared model.

16.5 False rejection

A false rejection satisfies $\text{Req}(z)$ but not $\text{Impl}(z)$. It may represent availability, completeness, or usability failure rather than unauthorized acceptance. The distinction matters because a repair that eliminates false acceptance by rejecting everything is not a useful overall security design.

16.6 Full conformance

Full extensional conformance requires

$$\forall z [\text{Impl}(z) \leftrightarrow \text{Req}(z)].$$

It combines acceptance-soundness with completeness. Many security investigations begin with the one-sided safety direction because it isolates unauthorized acceptance.

Chapter 17

Defensive Theorems

17.1 Hardening theorems

A hardening theorem states that a design change reduces the probability, reachability, or consequence of a named undesirable event under a fixed model.

17.2 Isolation theorems

Isolation theorems compare reachable authority or blast radius before and after a boundary is imposed. Theorem 4.1 is the simplest graph-theoretic example.

17.3 Containment theorems

Containment theorems may leave the probability of initial compromise unchanged while proving a smaller consequence set or loss bound.

17.4 Rate-control theorems

Rate-control theorems bound online opportunity over a declared horizon. They must not be exported to an offline verification model without additional premises.

17.5 Factor-composition theorems

Factor results formalize how conjunction, independence, fallback, and recovery affect the accepted event set. They should analyze the complete route structure rather than only the advertised primary factors.

17.6 Recovery-route theorems

A recovery theorem can show that strengthening primary authentication is insufficient when an unchanged recovery route remains sufficient and weaker under the same cost model.

17.7 Repair-preservation theorems

For vulnerability class X and metric m_X , define

$$D_2 \succ_X D_1$$

when $m_X(D_2) < m_X(D_1)$ and the theorem's declared non-regression constraints are satisfied. This notation is intentionally local to a metric and model; it is not a universal total order on security designs.

Part V

AML D Security

Chapter 18

Security as a Settlement Problem

18.1 Why binary prove-or-fail is too crude

Once security claims are formal, an investigation can end in several mathematically different ways. A claim may be proved, refuted by a checked witness, shown independent of the admitted theory in a declared metatheory, or rendered uninformative because the admitted assumptions are inconsistent. Failure to find any such certificate is a separate operational outcome: UNKNOWN.

18.2 P: proving the security property

Let c be the target security proposition. Role P searches for a certificate in a proof-certificate language $\mathcal{C}_P(c)$ that an independent verifier accepts as a proof of c in the declared environment.

18.3 R: producing a verified counterexample

Role R searches for a certificate of $\neg c$. For a universal conformance claim this may be a concrete event or trace z^* together with checker receipts establishing the predicates needed to show that z^* is a counterexample.

18.4 I: independence

Role I addresses a different question: whether c is independent of the admitted theory, in a specified metatheory capable of certifying that claim. Merely failing to prove or refute c is not independence.

18.5 C: inconsistency

Role *C* searches for a certificate that the admitted assumptions or security specification are inconsistent. In an inconsistent theory, ordinary derivability can become uninformative, so detecting inconsistency is a distinct settlement outcome.

18.6 UNKNOWN

If no accepted P, R, I, or C certificate is found within the declared resources, the operational result is UNKNOWN. UNKNOWN means “not settled under these conditions,” not “independent,” “safe,” or “false.”

18.7 Independent verification

Agent identity is provenance, not validity. Every settlement certificate must be accepted because its semantic content, dependencies, scope, and environment satisfy the independent checker, not because it came from the “right” role.

Chapter 19

BANK and FUTUREBANK

19.1 The security problem that motivates two stores

A reasoning system investigating security must consider unverified possibilities: suspected defects, candidate repairs, possible traces, tentative invariants, and alternative policies. Those objects are useful for planning but dangerous if they are silently treated as established facts. The architecture therefore needs an epistemic separation boundary before it needs AMLD terminology.

19.2 The BANK

At time t , let

$$B_t = \{\text{objects accepted by the declared verifier and retained as trusted reasoning state}\}.$$

The BANK is shared verified memory. Provenance should record how each object entered, but provenance does not replace verification.

19.3 The FUTUREBANK

Let

$$F_t = \{\text{unverified hypotheses, imagined traces, conditional consequences, and candidate repairs}\}.$$

The FUTUREBANK permits planning without contaminating B_t .

19.4 Quarantine before verification

An object may be useful in F_t even when it would be unsafe to assert in B_t . Search and simulation can condition on “suppose h ” without asserting h as a verified premise.

19.5 Promotion into the BANK

Let A_t be the set of candidate deposits proposed during a round and let $V(A_t)$ be the subset accepted by the verifier. The trusted update is

$$B_{t+1} = B_t \cup V(A_t).$$

The FUTUREBANK may evolve by search-specific rules, but those rules do not redefine the BANK gate.

19.6 Preventing speculative contamination

Theorem 19.1 (Verifier-gated noncontamination). *Suppose the only operation that adds a new semantic assertion to B_t is $B_{t+1} = B_t \cup V(A_t)$, and V returns only checker-accepted assertions. Then no assertion rejected or never examined by V enters the BANK through the modeled update rule.*

Proof. Every new element of $B_{t+1} \setminus B_t$ lies in $V(A_t)$ by definition of the update, hence is checker-accepted. $\square$

19.7 Why BANK/FUTUREBANK precedes P/R/I/C conceptually

The distinction between trusted and speculative information is a general security need. P/R/I/C are specialized search roles that become useful once the system asks which kinds of certificates can settle a formal security claim. The architecture is therefore motivated in the order: separation of epistemic status, verifier gate, then specialized settlement roles.

Chapter 20

Security Memory

20.1 Remembering successful defenses

Verified repairs and invariants should remain available so future reasoning does not repeatedly rediscover the same protection.

20.2 Remembering failed defenses

A failed repair is also information. It should not enter the BANK as a successful theorem, but its failure record can be retained as provenance-bearing search memory.

20.3 Rejected defect hypotheses

A rejected hypothesis may become useful negative evidence about a class of explanations. Retention must preserve its rejected status so later retrieval does not convert it into an established defect.

20.4 Negative knowledge

Negative knowledge includes tested approaches that failed, candidates rejected by the verifier, and repairs that violated another constraint. It can reduce repeated search cost without weakening the verifier boundary.

20.5 Regression memory

Every verified failure suitable for future testing can become a regression case. Ocean memory and reasoning memory therefore reinforce one another: one remembers how search failed; the other remembers an external test that future versions must survive.

20.6 Cross-system transfer

Transfer is useful only when provenance and applicability conditions are preserved. A repair that worked under one threat model should not be asserted under another merely because its surface form is similar.

20.7 Append-only security history

A simple memory law is

$$M_{t+1} = M_t \cup E_{t+1},$$

where E_{t+1} contains newly recorded evidence with outcome labels, provenance, environment, and verification status. “Append-only” protects history from silent erasure; it does not mean every old item remains equally relevant.

Chapter 21

AMLD Control and Repair

21.1 Security objectives

Control requires an objective vector: compromise probability, blast radius, availability, verification cost, latency, memory, and other constraints. A controller cannot optimize “security” until these quantities and priorities are stated.

21.2 Independent control variables

Policy thresholds, resource allocations, isolation choices, retry budgets, search novelty, and verification schedules can be modeled as control variables when the system is authorized to change them.

21.3 Dissatisfaction with a defense

A controller needs evidence that the current strategy is inadequate. Sustained verifier rejection, poor progress, repeated regression failure, or a dominated security vector can all serve as signals depending on the model.

21.4 Ordinary optimization

When a strategy appears productive, local refinement may be preferable to structural change. This is the security analogue of continuing to improve a defense whose measured objective is moving in the intended direction.

21.5 Structural revision

When local refinement repeatedly fails, the controller may consider a qualitatively different architecture: greater isolation, a different recovery rule, a new gate, or a revised factor composition. Such revision remains speculative until verified.

21.6 Comparing candidate repairs

Candidate repairs should be evaluated against both the target vulnerability and non-regression constraints. A repair that closes one acceptance path by denying all legitimate users is not automatically superior.

21.7 Verifier-gated deployment

The repair pipeline mirrors the BANK boundary:

candidate repair → analysis/Ocean → independent checks → deployment.

Planning may be creative; deployment authority remains narrow.

Part VI

Experiments and Research Frontier

Chapter 22

Security Experiments

22.1 What constitutes an experiment?

An experiment should name the system version, threat model, target property, data or problem family, resource budget, outcome criteria, and verifier. Without these, a success or failure is difficult to interpret or reproduce.

22.2 Threat-model declaration

Every experimental result is conditional on a model. The experiment should record which observations and actions are allowed and which components are trusted.

22.3 LAB, paired, holdout, and target cases

LAB cases support development. Paired cases compare controlled variants. Holdouts estimate generalization. Target cases are the questions ultimately of interest. Mixing these roles can produce optimistic estimates.

22.4 Security baselines

A new AMLD or Ocean mechanism should be compared with matched baselines: same verifier, same target, same resource accounting, and clearly stated differences in memory or control.

22.5 Ablations

Ablations remove one mechanism at a time: persistent failure memory, FUTUREBANK depth, cross-route BANK sharing, Ocean gating, or another feature. They help distinguish architectural

contribution from incidental complexity.

22.6 Preregistration

For high-value claims, the threat model, metric, test split, stopping rule, and analysis should be specified before the final target results are inspected.

22.7 Reproducibility and certificates

Raw logs are useful but not sufficient. Security experiments should preserve code version, configuration, environment, verifier output, and where possible independently checkable certificates for formal claims.

Chapter 23

Measuring Defensive Improvement

23.1 Compromise probability

Probability of unauthorized success is one metric, but only when the underlying distribution and episode are specified.

23.2 Blast radius

Blast radius measures consequence rather than initial probability. It can be represented as a reachable set, weighted loss, or another domain-specific damage quantity.

23.3 Availability cost

Controls can create denial or latency. Availability cost should therefore appear beside security gain when the defense trades one for the other.

23.4 Verification cost

Formal assurance consumes computation, engineering effort, and sometimes runtime latency. The verifier should remain independent of search, but its cost belongs in system evaluation.

23.5 Repair cost

A security design that is theoretically strong but operationally impossible to repair may perform poorly over time. Repair and recovery cost are therefore legitimate coordinates.

23.6 Multi-objective security

Let

$$\Delta S = (\Delta p_{\text{comp}}, \Delta |B|, \Delta A, \Delta C_V, \Delta C_R, \dots).$$

The sign convention should be stated so that improvement is unambiguous. The vector makes visible when a defense helps one objective and harms another.

23.7 Pareto frontiers

A design is Pareto-dominated if another design is no worse on all declared coordinates and strictly better on at least one. This is often a more honest comparison than forcing every objective into a single weighted sum.

Chapter 24

AMLD Security Conjectures

This chapter states research conjectures, not established theorems. Each should eventually be paired with a preregistered experimental design and an explicit class of systems for which it is claimed.

24.1 Audit-over-secret advantage

Conjecture 24.1. *For some realistic, well-rate-limited authentication classes with low top- N exposure, allocating AMLD-style search primarily to specification conformance, recovery routes, and control-policy auditing yields greater defensive value per unit computation than allocating the same resources primarily to direct secret-candidate ranking.*

24.2 Cross-route shared-BANK advantage

Conjecture 24.2. *For systems with several sufficient routes, a provenance-preserving shared BANK of verified invariants and counterexample lemmas can reduce expected audit cost relative to matched isolated route audits, especially for interaction defects.*

24.3 Adaptive flood-control frontier

Conjecture 24.3. *For some threat and usability distributions, an adaptive rate-control policy using verified account and risk state achieves a strictly better compromise-versus-denial tradeoff than every fixed hard-lockout policy in a declared comparison class.*

24.4 Verifier-history repair advantage

Conjecture 24.4. *Retaining rejected defect hypotheses and failed repair attempts outside the verified BANK but inside provenance-preserving search history can improve future repair selection without increasing false certification.*

24.5 Ocean-regression-moat advantage

Conjecture 24.5. *For some evolving ATP or AMLD systems, an accumulating Ocean suite that permanently retains verified regression cases reduces the rate of reintroduced verifier-boundary failures relative to a matched development process that discards historical failure cases.*

24.6 Separation-aware AMLD control

Conjecture 24.6. *A controller that explicitly includes blast radius and authority-boundary structure in its repair objective can find defenses with better containment properties than a matched controller optimizing only direct acceptance or proof-search success.*

24.7 Persistent-failure-memory advantage

Conjecture 24.7. *On some repeated security-audit distributions, append-only, outcome-labeled failure memory reduces expected repeated search cost while preserving verifier-gated correctness, relative to matched systems that retain only successes.*

Chapter 25

Claims We Should Not Make

25.1 No finite test proves universal security

A finite Ocean suite can exclude tested failures and estimate behavior on holdouts. It cannot quantify over every future program state, threat model, hardware defect, or environment unless a separate formal argument provides that quantification.

25.2 More searching is not necessarily safer

Additional search can consume resources, increase complexity, or generate more speculative state. Search helps only when the architecture preserves the verifier boundary and uses the additional work productively.

25.3 More reflection is not necessarily safer

Self-description and metareasoning can improve control in some systems and waste resources in others. No universal optimum follows from the architecture alone.

25.4 More isolation is not automatically better

Isolation can improve containment while harming availability, operability, or required communication. The goal is justified separation with narrow verified bridges, not maximum disconnection regardless of function.

25.5 Verification is only as meaningful as its specification

A perfectly correct checker can certify conformance to an inadequate or inconsistent specification. Specification review and contradiction detection therefore remain first-class tasks.

25.6 A secure component does not imply a secure composition

Interactions, shared state, recovery paths, and authority transfer can create system-level failures absent from component-local proofs.

25.7 UNKNOWN is not failure

UNKNOWN is an epistemic and operational status. It reports that no accepted settlement certificate was obtained under the declared conditions. Treating UNKNOWN as either proof of safety or proof of vulnerability would erase the distinction that motivated P/R/I/C in the first place.

Part VII

Back Matter

Appendix A

Glossary of Terms

Acceptance-soundness

The one-sided conformance property $\forall z[\text{Impl}(z) \Rightarrow \text{Req}(z)]$.

AMLD

In this manuscript, the verifier-gated multi-role logical-decider architecture built around settlement roles, shared verified memory, speculative planning, and explicit verification boundaries.

Attack surface

The set of externally or internally reachable interfaces and transitions relevant to the declared threat model. The term is descriptive; the manuscript focuses on reducing and controlling the surface rather than cataloging exploitation procedures.

Authorization

The policy decision that determines whether an established identity or context may perform an action in a state.

Authentication

The process by which presented evidence is evaluated against an authentication policy.

BANK

Shared memory containing assertions admitted only through the declared verification gate.

Blast radius

The set or weighted consequence of protected resources reachable after a modeled component compromise.

Certificate

A finite object whose validity for a declared claim can be checked by an independent verifier.

Compromise

A modeled loss of a declared security property, such as unauthorized authority or disclosure. The exact meaning must be stated per theorem.

Containment

Reduction of consequences after an undesirable event occurs.

Credential

Evidence or an object used by a policy to establish authority or identity claims.

FUTUREBANK

Quarantined speculative state containing hypotheses, imagined traces, conditional consequences, and candidate repairs not yet admitted to the BANK.

Invariant

A property intended to hold in every reachable state of a model.

Least privilege

The design principle of granting only authority required for the declared function and duration.

Ocean

In this treatise, the ATP test environment used as a verifier-gated regression moat for candidate reasoning-system versions.

Refutation

A checked certificate that a target proposition is false in the declared environment.

Regression moat

An accumulating suite of verified historical failure cases that a future candidate must survive before admission.

Security domain

A collection of state and authority governed by a common policy boundary.

Threat model

The declared assets, adversary observations and actions, resources, trusted components, and environmental assumptions under which a security claim is evaluated.

Verifier

The independent checker that determines whether a proposed certificate, boundary transition, or other formally specified object satisfies the declared acceptance rule.

UNKNOWN

The result returned when no accepted P/R/I/C settlement certificate is obtained under the declared search conditions.

Appendix B

Glossary of Symbols and Notation

The “classical reference” column points to a standard book in which the underlying notation or mathematical object is developed. It does not imply that book-specific AMLD notation appears in that source. The “GitHub/source reference” column gives a stable LaTeX label in this draft. Before publication, these labels should be paired with a frozen repository path and commit hash; line-number links should be generated only after the source is committed because line numbers move during editing.

Symbol	Meaning here	Classical book	GitHub/source reference
$\in$	Membership: $x \in A$ means x is an element of A .	Enderton, <i>Elements of Set Theory</i> .	Draft source; set notation used throughout; see label <code>ch:separation</code> .
$\notin$	Non-membership.	Enderton, <i>Elements of Set Theory</i> .	Same source; standard notation.
$\subseteq$	Subset or inclusion.	Enderton, <i>Elements of Set Theory</i> .	Label <code>sec:blast-radius</code> ; Theorem 4.1.
$\not\subseteq$	Not a subset: inclusion fails.	Enderton, <i>Elements of Set Theory</i> .	Same source; glossary definition.
$\subsetneq$	Proper subset.	Enderton, <i>Elements of Set Theory</i> .	Label <code>sec:blast-radius</code> ; strict containment discussion.
$\cup$	Set union.	Enderton, <i>Elements of Set Theory</i> .	Labels <code>sec:ocean-regression</code> , <code>sec:append-memory</code> .
$\cap$	Set intersection.	Enderton, <i>Elements of Set Theory</i> .	Used in domain and authority separation discussions.
$\setminus$	Set difference.	Enderton, <i>Elements of Set Theory</i> .	Used in proofs involving newly added BANK elements.
$\emptyset$	Empty set.	Enderton, <i>Elements of Set Theory</i> .	Standard notation; available for later revisions.

Symbol	Meaning here	Classical book	GitHub/source reference
$\neg P$	Logical negation.	Enderton, <i>A Mathematical Introduction to Logic</i> .	Label <code>sec:acceptance-soundness</code> .
$P \wedge Q$	Logical conjunction.	Enderton, <i>A Mathematical Introduction to Logic</i> .	Chapter <code>ch:factors</code> .
$P \vee Q$	Logical disjunction.	Enderton, <i>A Mathematical Introduction to Logic</i> .	Standard notation; used as needed in specifications.
$P \Rightarrow Q$	Material implication in the formal claims of this book.	Enderton, <i>A Mathematical Introduction to Logic</i> .	Labels <code>sec:verify-verifier</code> , <code>sec:acceptance-soundness</code> .
$P \leftrightarrow Q$	Biconditional.	Enderton, <i>A Mathematical Introduction to Logic</i> .	Chapter <code>ch:conformance</code> , full conformance.
$\forall x$	Universal quantifier.	Enderton, <i>A Mathematical Introduction to Logic</i> .	Label <code>sec:acceptance-soundness</code> .
$\exists x$	Existential quantifier.	Enderton, <i>A Mathematical Introduction to Logic</i> .	Counterexample discussions in Chapters <code>ch:specifications</code> and <code>ch:conformance</code> .
$\Pr(E)$	Probability of event E .	Feller, <i>An Introduction to Probability Theory and Its Applications</i> , Vol. I.	Chapter <code>ch:information</code> .
$\Pr(X = x \mid Z = z)$	Conditional probability after side information.	Feller, <i>An Introduction to Probability Theory and Its Applications</i> , Vol. I.	Label <code>sec:observer-info</code> ; Chapter <code>ch:secrets</code> .
$H(X)$	Shannon entropy when used.	Cover and Thomas, <i>Elements of Information Theory</i> .	Chapter <code>ch:information</code> .
H_∞	Min-entropy.	Cover and Thomas for information-theory background; modern cryptography texts for min-entropy use.	Chapter <code>ch:prob-compromise</code> .
$G = (V, E)$	Directed or undirected graph with vertices and edges, as declared.	Diestel, <i>Graph Theory</i> .	Label <code>sec:blast-radius</code> .
$\text{Reach}_G(c)$	Vertices reachable from c in graph G .	Diestel, <i>Graph Theory</i> for reachability background.	Label <code>sec:blast-radius</code> ; defined in this treatise.
$B_G(c)$	Blast radius of component c in graph G .	Book-specific notation; graph background in Diestel.	Label <code>sec:blast-radius</code> ; defined here.

Symbol	Meaning here	Classical book	GitHub/source reference
$G_N(\pi)$	Top- N probability mass.	Book-specific notation; probability background in Feller.	Label <code>sec:top-n</code> ; defined here.
$\text{Impl}(z)$	Implemented acceptance predicate.	Book-specific notation; logic background in Enderton.	Labels <code>sec:verify-verifier</code> , <code>sec:acceptance-soundness</code> .
$\text{Req}(z)$	Required or intended acceptance predicate.	Book-specific notation; logic background in Enderton.	Labels <code>sec:verify-verifier</code> , <code>sec:acceptance-soundness</code> .
P, R, I, C	Prover, Refuter, Independence, and Contradiction settlement roles.	AMLD-specific notation defined in this treatise.	Chapter <code>ch:settlement</code> .
B_t	Verified BANK contents at time t .	AMLD-specific notation defined here; set background in Enderton.	Label <code>sec:bank-definition</code> .
F_t	FUTUREBANK speculative contents at time t .	AMLD-specific notation defined here; set background in Enderton.	Label <code>sec:futurebank-definition</code> .
M_t	Provenance-bearing security/search memory at time t .	Book-specific notation.	Label <code>sec:append-memory</code> .
$\mathcal{O}_t$	Ocean regression-test collection at time t .	Book-specific notation.	Labels <code>ch:ocean</code> , <code>sec:ocean-regression</code> .
$D_2 \succ_X D_1$	Defense D_2 strictly improves the declared metric for vulnerability class X subject to stated non-regression constraints.	Book-specific notation; order-relation background in standard set/order texts.	Label <code>sec:defense-dominance</code> .
UNKNOWN	No accepted P/R/I/C settlement certificate was obtained under declared conditions.	AMLD-specific notation.	Label <code>sec:unknown</code> .

Appendix C

Draft Source and GitHub Reference Map

This draft has not yet been committed to a repository. To avoid inventing live links, the manuscript uses source-file and LaTeX-label references rather than URLs that might not exist. A publication commit should freeze the following mapping:

- Source file: `AMLD_Security_Treatise_v0_1.tex`.
- Suggested repository-relative path: `manuscripts/security/AMLD_Security_Treatise_v0_1.tex`.
- Each symbol-glossary entry points to a LaTeX label such as `sec:blast-radius` or `sec:bank-definition`.
- After commit, replace or supplement these with a permanent GitHub URL containing the repository, commit hash, file path, and line range. A commit hash is preferable to a link to `main` because the reference should continue to identify the exact edition read by the user.

Appendix D

References

Bibliography

- [1] Ross Anderson. *Security Engineering: A Guide to Building Dependable Distributed Systems*. 3rd ed. Wiley, 2020.
- [2] Matt Bishop. *Computer Security: Art and Science*. 2nd ed. Addison-Wesley, 2018.
- [3] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd ed. Wiley, 2006.
- [4] Reinhard Diestel. *Graph Theory*. Springer. Edition to be fixed in the publication bibliography.
- [5] Herbert B. Enderton. *Elements of Set Theory*. Academic Press, 1977.
- [6] Herbert B. Enderton. *A Mathematical Introduction to Logic*. 2nd ed. Academic Press, 2001.
- [7] William Feller. *An Introduction to Probability Theory and Its Applications*, Vol. I. Wiley. Edition details to be fixed in the publication bibliography.
- [8] National Institute of Standards and Technology. *Digital Identity Guidelines*, Special Publication 800-63 series. Exact edition and retrieval date to be verified before publication.
- [9] OWASP Foundation. Authentication and application-security guidance. Exact pages, versions, and retrieval dates to be verified before publication.
- [10] Jerome H. Saltzer and Michael D. Schroeder. “The Protection of Information in Computer Systems.” *Proceedings of the IEEE* 63(9), 1975, 1278–1308.
- [11] Brian Tenneson. *What Checks the Proof?* AMLD Textbook v6.80. Working research manuscript, August 22, 2026. Internal program reference for the earlier authentication application and research-conjecture lineage.

Draft Status

This is a structural first draft. The chapter sequence, defensive theorem policy, Ocean regression-moat concept, motivation for BANK/FUTUREBANK, motivation for P/R/I/C, glossary architecture, and source-reference architecture are intended for review. Several chapters deliberately state modest propositions rather than ambitious claims. External reference metadata, GitHub commit links, and any theorem whose assumptions are strengthened in later drafts should be rechecked before release.